Laboratory 7:

Fourier Series & Linear Systems
Music Synthesizer: Building Sound from Harmonics

Proof of Concept: Harmonic Additive Synthesis

Lab Duration: 2 hours
Pre-Lab Time Expected: 45 minutes (Fourier coefficient calculations)
Key Concept: Any periodic sound can be synthesized by summing pure sine waves (harmonics) at specific frequencies and amplitudes. This lab proves that hypothesis on real hardware—and you'll hear it work.

Contents

Grading and Authenticity Requirements

Deliverable (1/2): This is a graded laboratory assignment. Include your **name and student ID** on all handwritten work, screenshots, and audio recordings.

Deliverable (2/2): All Fourier coefficient calculations must be **handwritten and shown to the TA before you implement code**. Pre-computed coefficients from online tools are not accepted; you must derive them yourself.

Checkpoint (1/7): Before submission, verify that each MATLAB figure includes axis labels with units, a legend, your name/student ID in the title, and a clear reference to your calculated coefficients.

1 Real-World Context: Harmonic Synthesis Across Audio Industries

You are a Junior Audio Engineer at a rapidly growing audio technology company. Your department designs signal processing systems for music production, entertainment, and biomedical applications. Your expertise in Fourier Series and harmonic synthesis opens doors across three very different programs—each relying on the same mathematical principles you'll explore in this lab.

1.1 Program 1: Hardware Synthesizers for Live Music (Music Industry)

Your company is developing a new hardware synthesizer for professional musicians and producers. The flagship feature: real-time waveform morphing controlled by physical knobs and touch pads on stage.

The challenge: A live performer in a packed concert hall cannot afford latency or stuttering. When they turn a knob to morph from a sine wave to a square wave, the change must happen within 5 milliseconds. Their expectation: infinite harmonic richness, morphed instantly on affordable hardware (a microcontroller, not a GPU rack).

Your solution: Prove that harmonic synthesis—generating arbitrary waveforms by summing controlled sine and cosine harmonics—delivers on this promise. Your lab demonstrates how to hand-calculate Fourier Series coefficients, implement them in C++ on an Arduino, and hear the real-time results. The spectral purity matters: a single misplaced harmonic or quantization error ruins the timbre and alienates users.

Key performance metrics: zero latency, spectral accuracy $\pm 2\%$ in harmonic amplitude, seamless morphing (no clicks or zipper noise), and robust operation across concert venues (temperature variations, power supply noise).

1.2 Program 2: Film and Game Audio Design (Entertainment)

Your company is contracting with major game studios and film scoring teams to create a next-generation orchestral synthesis engine. The goal: allow composers to blend synthetic and sampled instruments in real-time, creating custom hybrid sounds.

The challenge: A film composer working against a tight deadline needs to adjust the timbre of a virtual violin or trumpet mid-scene. Traditional approach: resample pre-recorded instruments. Problem: that requires massive storage (gigabytes of audio files) and inflexible edits.

Your solution: Use Fourier Series to decompose recorded instrument samples into their harmonic components, allowing composers to manipulate the harmonics (suppress a nasal tone by reducing the 3rd harmonic, brighten by boosting the 4th). Your lab proves the feasibility: harmonic reconstruction must achieve $> 95\%$ perceptual quality (measured by ABX listening tests) to avoid studio rejection.

Key performance metrics: imperceptible reconstruction quality, real-time harmonic editing, and smooth transitions between harmonic sets (no audible discontinuities).

1.3 Program 3: Hearing Aids and Auditory Prosthetics (Biomedical)

Your company partners with a leading hearing aid manufacturer to develop personalized hearing restoration. Modern hearing loss is frequency-selective—a user may lose sensitivity in the 2–4 kHz band but hear low frequencies normally.

The challenge: Generic hearing aid amplification is crude and unnatural. Patients complain of "roboticness" and fatigue. Can we synthesize missing harmonics from other frequency bands to restore natural-sounding speech and music?

Your solution: Use Fourier analysis to decompose incoming speech and music into harmonics. For frequencies the user cannot hear, synthesize them from neighboring audible harmonics—using harmonic relationships to fill in the gap. Your lab is the proof-of-concept: demonstrating that controlled harmonic synthesis can reconstruct missing spectral content without artifacts. Clinical trials require $\geq 15\%$ improvement in speech intelligibility and subjective naturalness (measured by patient surveys).

Key performance metrics: imperceptible latency (< 5 ms), no harmonic distortion artifacts, robust across speech and music, and safe for continuous 8-hour daily use.

1.4 The Common Thread: Understanding Harmonics

All three programs—concert synthesizers, film orchestration, hearing restoration—depend on a single core skill: **understanding how to decompose and reconstruct sounds using Fourier Series**. In this lab, you will:

1. Hand-calculate Fourier coefficients for a periodic waveform.
2. Synthesize that waveform by summing controlled harmonics on real hardware.
3. Validate your implementation against spectral theory using FFT analysis.
4. Observe the Gibbs Phenomenon—the ringing artifacts when truncating the infinite series.
5. Discuss real-world trade-offs: how many harmonics are "enough," when does quantization error become audible, and what latency constraints matter for each application.

2 Learning Objectives

By the end of this lab, you should be able to:

1. **Derive Fourier Series coefficients:** Hand-calculate the trigonometric or exponential Fourier Series representation for a periodic waveform assigned by the TA.
2. **Predict spectral content:** Given your coefficients, sketch the magnitude spectrum (magnitude of each harmonic) and predict how many harmonics are needed to approximate the target waveform.
3. **Translate mathematics to code:** Implement harmonic synthesis in C++ by computing weighted sine waves at multiples of the fundamental frequency and summing them.
4. **Control synthesis interactively:** Add harmonics one at a time using a button press, hearing and seeing the waveform evolve on oscilloscope and speaker.
5. **Validate with spectral analysis:** Use MATLAB's FFT to compute the empirical frequency spectrum of your hardware output and compare with hand-calculated predictions.
6. **Understand the Gibbs Phenomenon:** Observe and explain why truncating the Fourier Series at a finite number of terms creates ringing and overshoot at signal discontinuities.
7. **Appreciate hardware constraints:** Discuss trade-offs between signal fidelity, computational power, sampling rate, and memory on a resource-limited microcontroller.

3 Fourier Series: The Mathematical Foundation

3.1 Trigonometric Fourier Series

Any periodic signal $x(t)$ with period T can be represented as an infinite sum of sines and cosines:

$$x(t) = \frac{a_0}{2} + \sum_{k=1}^{\infty} \left[a_k \cos(k\omega_0 t) + b_k \sin(k\omega_0 t) \right], \tag{1}$$

where:

- $\omega_0 = 2\pi/T$ is the fundamental angular frequency (rad/s).
- $k = 1, 2, 3, \ldots$ indexes the harmonics (integer multiples of the fundamental).
- $a_0/2$ is the DC component (average value).
- a_k and b_k are the Fourier coefficients, computed via integration.

Computation of coefficients:

$$a_k = \frac{2}{T} \int_0^T x(t) \cos(k\omega_0 t)\, dt, \quad k = 0, 1, 2, \ldots \tag{2}$$

$$b_k = \frac{2}{T} \int_0^T x(t) \sin(k\omega_0 t)\, dt, \quad k = 1, 2, 3, \ldots \tag{3}$$

3.2 Exponential (Complex) Fourier Series

Alternatively, using Euler's formula $e^{j\theta} = \cos\theta + j\sin\theta$:

$$x(t) = \sum_{k=-\infty}^{\infty} c_k e^{jk\omega_0 t}, \tag{4}$$

where the complex coefficient is:

$$c_k = \frac{1}{T} \int_0^T x(t) e^{-jk\omega_0 t}\, dt. \tag{5}$$

Relationship to trigonometric form:

$$a_k = c_k + c_{-k}^* \text{ (real part)} \tag{6}$$

$$b_k = j(c_k - c_{-k}^*) \text{ (imaginary part)} \tag{7}$$

$$|c_k| = \frac{1}{2}\sqrt{a_k^2 + b_k^2} \quad \text{(magnitude spectrum)} \tag{8}$$

3.3 From Fourier Series to Synthesis (The Inverse Problem)

Analysis: Given $x(t)$, compute a_k, b_k (or c_k).

Synthesis (your task): Given a_k, b_k (or c_k), reconstruct $x(t)$ by summing harmonics.

In this lab, you will:

1. Hand-calculate the Fourier coefficients for your assigned waveform.

2. Store these coefficients in your Arduino program.
3. Generate discrete-time samples by evaluating the Fourier Series at sampled times: $x[n] = \frac{a_0}{2} + \sum_{k=1}^{N} [a_k \cos(k\omega_0 n T_s) + b_k \sin(k\omega_0 n T_s)]$, where $T_s = 1/f_s$ is the sampling period.
4. Send those samples to the DAC (Digital-to-Analog Converter), creating an analog waveform.
5. Add harmonics one at a time ($N = 1, 2, 3, \ldots$) to watch the waveform converge to your target.

3.4 The Gibbs Phenomenon

When you truncate the Fourier Series to a finite number of harmonics ($N < \infty$), the reconstructed signal exhibits **overshoot and ringing** near discontinuities. This is the **Gibbs Phenomenon**.

Key insight: No matter how many harmonics you add, overshoot persists at about 9% of the discontinuity magnitude. But the width of the overshoot region shrinks as N increases.

In this lab, you will **observe** the Gibbs Phenomenon as you add harmonics one at a time—hearing the buzzing/ringing sound at edges, and seeing it on the oscilloscope.

[INSERT DIAGRAM: Gibbs Phenomenon illustration (square wave approximation with N=1, 3, 5, 9 harmonics, showing convergence and overshoot)]

4 Pre-Lab Preparation (Your Assignment)

4.1 Your Target Waveform

The TA will assign you **one unique waveform** from the following list:

- **Square Wave:** $x(t) = \begin{cases} +1 & 0 < t < T/2 \\ -1 & T/2 < t < T \end{cases}$ (50% duty cycle)
- **Sawtooth Wave:** $x(t) = \frac{2t}{T} - 1$, for $0 < t < T$ (ramps linearly from -1 to $+1$)
- **Triangular Wave:** $x(t) = \begin{cases} \frac{4t}{T} - 1 & 0 < t < T/2 \\ 3 - \frac{4t}{T} & T/2 < t < T \end{cases}$ (piecewise linear peaks)
- **Pulse Wave (adjustable duty cycle):** $x(t) = \begin{cases} +1 & 0 < t < \alpha T \\ -1 & \alpha T < t < T \end{cases}$, where α is specified by TA (e.g., $\alpha = 0.3$ gives 30% duty cycle)
- **Half-Wave Rectified Sine:** $x(t) = \begin{cases} \sin(\omega_0 t) & 0 < t < T/2 \\ 0 & T/2 < t < T \end{cases}$

Student Notes: Your assigned waveform (fill in during lab):

Waveform type: _____

Any parameters (e.g., duty cycle, frequency): _____

4.2 Hand-Calculation of Fourier Coefficients

Pre-lab Deliverable (1/4): You must hand-derive the Fourier coefficients for your assigned waveform. Show all integration steps. Use either the trigonometric form (a_k, b_k) or the exponential form (c_k); your TA will accept either.

Student Notes: Space for handwritten Fourier Series derivation:

4.3 Predicted Magnitude Spectrum

Pre-lab Deliverable (2/4): Using your calculated coefficients, fill in the magnitude of the first 10 harmonics.

Table 1. Predicted Magnitude Spectrum

k	0	1	2	3	4
$\lvert a_k\rvert$ or $2\lvert c_k\rvert$	_____	_____	_____	_____	_____

Table 2. (Continued)

k	5	6	7	8	9
$\lvert a_k\rvert$ or $2\lvert c_k\rvert$	_____	_____	_____	_____	_____

Student Notes: Observations about your spectrum:

4.4 Sampling Rate and Number of Harmonics

The Arduino will synthesize your waveform by evaluating the Fourier Series at sampled times. Choose:

- **Fundamental frequency** f_0 (Hz): _____ (typical: 100–1000 Hz for audio)
- **Sampling rate** f_s (Hz): _____ (must be $\geq 2 \times 10 f_0$ to capture 10 harmonics per Nyquist)
- **Number of harmonics** N: _____ (plan to start with $N = 5$ or $N = 10$)

Student Notes: Sample count per period: $N_{\text{samples}} = f_s/f_0 =$ _____

(This is how many samples you'll need to store one period of the synthesized waveform)

Checkpoint (2/7): Show the TA your handwritten derivations, coefficient table, and sampling plan for Checkpoint 1 sign-off.

5 Required Equipment and Software

Table 3. Required Hardware and Software

Category	**Required Items**
Microcontroller	Arduino Uno or Mega, USB cable
Digital-to-Analog	External R-2R DAC or PWM-filtered output (pre-amp designed breadboard)
Audio Output	Resistive or magnetic speaker (8 Ω), audio amplifier circuit (Op-amp or ready-made module)
Control	Push button on D2 (for adding harmonics one at a time)
Measurement	Benchtop oscilloscope (10 MHz+), audio headphones or speaker
Prototyping	Breadboard, jumper wires, capacitors (10 µF, 100 nF), resistors
Software	Arduino IDE, MATLAB R2021a or later
Provided skeletons	`HarmonicSynthesis.ino`, `Lab07_WaveformCapture.m`

Arduino Pin Roles

Table 4. Pin Assignments

Signal	**Pin**	**Direction**	**Purpose**
DAC Output	D11 (PWM)	OUTPUT	Sends synthesized waveform (filtered to analog)
Harmonic Increment Button	D2	INPUT	Press to add next harmonic ($N \leftarrow N + 1$)
Oscilloscope Probe	Analog Out	INPUT (measure)	Monitor synthesized waveform
Serial TX	USB	TX	Stream waveform data to MATLAB

6 Hardware Setup: The Synthesis Chain

6.1 System Overview

1. **Arduino PWM/DAC**: Generates a digital representation of your synthesized signal (8-bit or 12-bit).
2. **Low-pass filter**: Smooths the digital output to a continuous analog waveform (removes high-frequency switching artifacts).
3. **Audio amplifier**: Boosts the weak analog signal to speaker-level (typically 1–2 V at 100 mA).
4. **Speaker**: Converts electrical signal to sound.
5. **Oscilloscope probe**: Measures the analog waveform after filtering (but before amplification, to avoid loading effects).

6.2 PWM-to-Analog Conversion

The Arduino Uno does not have a true DAC. Instead, it uses Pulse-Width Modulation (PWM) on pin D11 at 490 Hz (configurable). By varying the duty cycle (how long the output is HIGH vs. LOW in each PWM period), you can effectively generate 256 discrete voltage levels between 0 and 5 V.

How it works:

$$V_{\text{out}} = V_{\text{supply}} \times \frac{\text{PWM duty cycle}}{255}. \tag{9}$$

A low-pass filter (typically RC circuit with cutoff frequency ~ 1 kHz) removes the 490 Hz PWM carrier and leaves only the slow-moving signal envelope—your synthesized waveform.

[INSERT AUDIO AMPLIFIER AND LOW-PASS FILTER WIRING SCHEMATIC: Arduino D11 PWM to RC filter (e.g., 10kΩ + 10µF) to op-amp non-inverting amplifier to speaker, with oscilloscope probe shown after filter]

Student Notes: Student hardware checklist:

- ☐ RC low-pass filter wired to D11, cutoff ≈ 1 kHz
- ☐ Op-amp audio amplifier circuit powered and wired
- ☐ Speaker connected to amplifier output
- ☐ Oscilloscope probe on the analog signal (after filter)
- ☐ Button connected to D2 and GND

Checkpoint (3/7): TA visually inspects the filter and amplifier circuits. Verify that a DC voltage from the Arduino (using `analogWrite()` test) produces a corresponding DC voltage at the amplifier output.

7 Lab Procedure (In-Lab Execution, 2 Hours)

Target Timeline: CP-1 (10 min) → Hardware check (10 min) → Arduino coding and testing (40 min) → Button control and harmonic stepping (20 min) → MATLAB capture and spectral analysis (30 min) → wrap-up (10 min).

7.1 Part 1: Pre-Lab Verification and Hardware Check

1. Show the TA your handwritten Fourier coefficient derivation, predicted spectrum table, and sampling plan.
2. Power up the breadboard and Arduino.
3. Using the oscilloscope, verify that a simple DC test signal produces expected voltage at the amplifier output (use `analogWrite(D11, 128)` to output 2.5 V).
4. Verify that sound is audible from the speaker when the amplifier is driven.

Checkpoint (4/7): TA signs off on pre-lab work and confirms hardware functionality for Checkpoint 1 (CP-1).

7.2 Part 2: Arduino Harmonic Synthesis Implementation

Objective: Program the Arduino to generate your target waveform by summing harmonics according to your calculated Fourier coefficients.

Skeleton Code Overview

The provided `HarmonicSynthesis.ino` skeleton contains:

- A pre-defined array of samples (one period of the target waveform, computed offline or via your calculated Fourier coefficients).
- A high-speed timer interrupt that reads samples from this array and outputs them to the PWM pin at a precise sampling rate.
- A button interrupt handler that increments the number of active harmonics.

You must fill in:

1. Your calculated Fourier coefficients (a_k, b_k, or c_k) as constants in the code.
2. The loop that generates the sample array by evaluating:

$$x[n] = \frac{a_0}{2} + \sum_{k=1}^{N} [a_k \cos(2\pi kn/N_{\text{samples}}) + b_k \sin(2\pi kn/N_{\text{samples}})] \tag{10}$$

for $n = 0, 1, \ldots, N_{\text{samples}} - 1$.

Configuration and Coefficient Entry

```
// USER CONFIGURATION
#define F0_HZ          500           // fundamental frequency (Hz)
#define FS_HZ          10000         // sampling rate (Hz)
#define N_SAMPLES      (FS_HZ / F0_HZ)  // samples per period
#define MAX_HARMONICS  10            // max number of harmonics to compute
```

```
// Your Fourier coefficients (fill in from pre-lab)
const float a0 = 0.0;              // DC component (or 2*c0 for exponential form)
const float a[] = {0.0, 1.27, 0.0, 0.42, 0.0, 0.254, ...};  // cosine coefficients
const float b[] = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...};      // sine coefficients
```

Student Notes: Your Fourier coefficients (transcribed from pre-lab):

$a_0 =$ _____

$a_1 =$ _____, $b_1 =$ _____

$a_2 =$ _____, $b_2 =$ _____

(continue for all 10 harmonics)

Generating the Sample Array

In the `setup()` function, compute the sample array:

```
void setup() {
  Serial.begin(115200);

  // Generate one period of the waveform
  for (int n = 0; n < N_SAMPLES; n++) {
    float sample = a0 / 2.0;  // DC component

    // Add harmonics up to the current number (num_harmonics)
    for (int k = 1; k <= num_harmonics; k++) {
      float harmonic_phase = (2.0 * PI * k * n) / N_SAMPLES;
      sample += a[k] * cos(harmonic_phase);
      sample += b[k] * sin(harmonic_phase);
    }

    // Scale to 0--255 PWM range (assuming signal is normalized to [-1, 1])
    uint8_t pwm_value = (uint8_t)(127.5 + 127.5 * sample);
    waveform_samples[n] = pwm_value;
  }

  // Start the timer interrupt to output samples at FS_HZ rate
  startTimerInterrupt(FS_HZ);
}
```

The skeleton code handles the timer interrupt and serial output. You only need to fill in the loop that computes `waveform_samples[]`.

Student Notes: Student notes on synthesis code:

7.3 Part 3: Button Control and Harmonic Stepping

Objective: Add harmonics one at a time when the button is pressed, hearing and seeing the waveform evolve.

The skeleton code includes a button interrupt on D2. When the button is pressed:

1. Increment `num_harmonics` (up to `MAX_HARMONICS`).
2. Recompute the sample array with the new `num_harmonics`.
3. Output is automatically re-generated at the next timer interrupt.

```
// Button interrupt handler
void buttonISR() {
  num_harmonics++;
  if (num_harmonics > MAX_HARMONICS) {
    num_harmonics = MAX_HARMONICS;
  }

  // Regenerate waveform with new harmonic count
  regenerateWaveform(num_harmonics);
}
```

Testing

1. Upload the code to the Arduino.
2. Open the oscilloscope and point it at the analog waveform (after the low-pass filter).
3. Initially, the waveform will show only the fundamental ($N = 1$): a single sinusoid.
4. Press the button repeatedly. With each press:
   - The waveform shape changes on the oscilloscope (visible Gibbs phenomenon: overshoot, ringing).
   - The tone changes—you should hear the timbre/pitch of the tone shift (higher harmonics add more "buzzing" or "sharpness").
5. Continue pressing until you reach $N = 10$ harmonics. Listen for the tone settling to a more stable character.

[INSERT EXPECTED OSCILLOSCOPE WAVEFORMS: Show N=1 (sine), N=3 (approximation with ringing), N=5, N=9 (much closer to target)]

Checkpoint (5/7): Demonstrate the working synthesizer to the TA: show the waveform changing on the oscilloscope as harmonics are added, and have the TA listen to the evolving tone from the speaker.

Student Notes: Observations while adding harmonics:

7.4 Part 4: Audio/Video Recording of Harmonic Addition

Pre-lab Deliverable (3/4): Using your phone's camera, record a 30-second video showing:

1. The oscilloscope screen displaying the waveform.
2. You pressing the button to add harmonics.
3. Audio of the tone changing (the microphone should capture the speaker output).

Save this video with your name/student ID in the filename. You will submit it as part of your deliverables.

Checkpoint (6/7): Show the TA the recorded video during Checkpoint 2 (CP-2).

7.5 Part 5: MATLAB Spectral Analysis and Validation

Objective: Capture the actual waveform output from your Arduino, compute the empirical FFT (frequency spectrum), and compare it to your hand-calculated Fourier coefficients.

Task 1: Capture Waveform via Serial

The skeleton `Lab07_WaveformCapture.m` opens the Arduino serial port, triggers a capture of one complete period of your synthesized waveform, and imports the sample array.

```
% MATLAB: Connect to Arduino and capture
port = serialport("COM3", 115200);  % adjust COM port as needed
write(port, "c");                   % trigger capture command
data = readline(port);              % read back the samples
```

```
samples = str2num(data);            % convert to numeric array
```

Student Notes: MATLAB connection notes:
COM port: _____ Baud rate: _____

Number of samples captured: _____

Task 2: Compute and Plot FFT

Use MATLAB's FFT to extract the frequency spectrum:

```
% Compute FFT
Y = fft(samples);
N = length(samples);
frequencies = (0:N-1) * Fs / N;  % frequency axis (Hz)
magnitude = abs(Y) / N;           % magnitude spectrum

% Plot only positive frequencies up to Nyquist
figure;
plot(frequencies(1:N/2), magnitude(1:N/2), 'b-', 'LineWidth', 1.5);
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('Magnitude', 'FontSize', 12);
title(sprintf('Alice Smith (A123) - Empirical FFT Spectrum'), 'FontSize', 12);
grid on;
xlim([0, max(frequencies(1:N/2))]);
```

Create a figure showing the empirical FFT magnitude spectrum. Mark the peaks corresponding to each harmonic.

[INSERT EXAMPLE MATLAB FFT PLOT: Showing peaks at f0, 3*f0, 5*f0, etc. for a square wave, with magnitude decreasing for higher harmonics]

Task 3: Overlay Hand-Calculated Spectrum

On the same figure (or a side-by-side comparison), plot your hand-calculated Fourier coefficients as a stem plot:

```
% Hand-calculated coefficients
a_theory = [0.0, 1.27, 0.0, 0.42, 0.0, 0.254, ...];  % your coefficients
```

```
b_theory = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...];

% Magnitude: sqrt(a_k^2 + b_k^2)
magnitude_theory = sqrt(a_theory.^2 + b_theory.^2);

% Plot on the same figure
hold on;
stem(1:10, magnitude_theory(1:10), 'r*', 'MarkerSize', 12, 'DisplayName', 'Theory');
legend('Empirical FFT', 'Hand-Calculated');
```

Student Notes: Comparison notes:
Do the empirical FFT peaks align with your predicted magnitudes?

Task 4: Visualize Gibbs Phenomenon (Time-Domain)

Create a second figure showing the synthesized waveform with increasing numbers of harmonics:

```
% Generate partial-sum waveforms with N = 1, 3, 5, 10 harmonics
figure('Position', [100, 100, 1200, 800]);

for N_plot = [1, 3, 5, 10]
    subplot(2, 2, find([1, 3, 5, 10] == N_plot));

    % Synthesize waveform with N_plot harmonics
    x_partial = a0/2 * ones(size(t_samples));
    for k = 1:N_plot
        x_partial = x_partial + a_theory(k) * cos(k * omega0 * t_samples) ...
                              + b_theory(k) * sin(k * omega0 * t_samples);
    end

    % Plot
    plot(t_samples * 1000, x_partial, 'b-', 'LineWidth', 1.5);  % time in ms
    xlabel('Time (ms)', 'FontSize', 11);
    ylabel('Amplitude', 'FontSize', 11);
    title(sprintf('N = %d harmonics', N_plot), 'FontSize', 11);
    grid on;
    ylim([-1.5, 1.5]);  % show overshoot
end

sgtitle(sprintf('Alice Smith (A123) - Gibbs Phenomenon: Truncated Fourier Series'), ...
        'FontSize', 12);
```

This figure clearly shows:

- With $N = 1$: Only the smooth fundamental sine wave (no discontinuities yet).
- With $N = 3, 5$: Visible ringing and overshoot at discontinuities (Gibbs Phenomenon).
- With $N = 10$: Much sharper approximation to the target shape, but overshoot still present.

[INSERT GIBBS PHENOMENON MATLAB PLOT: Four subplots showing square wave approximation with N=1, 3, 5, 10 harmonics, clearly showing ringing and overshoot]

Checkpoint (7/7): Show TA all three MATLAB figures (FFT spectrum, overlay with theory, Gibbs phenomenon visualization) for Checkpoint 3 (CP-3).

Student Notes: Student observations on Gibbs Phenomenon:

8 Post-Lab Analysis and Deliverables

8.1 Required Discussion Questions

Pre-lab Deliverable (4/4): Answer each question in 3–6 complete sentences, supported by observations from your lab data.

1. **Spectrum Prediction vs. Reality:** Compare your hand-calculated Fourier coefficients with the empirical FFT spectrum from MATLAB. Where do they agree well, and where do they diverge? What could cause discrepancies (consider: ADC noise, rounding errors in the Arduino, sampling rate limitations)?
2. **Gibbs Phenomenon and Its Origin:** Explain why, no matter how many harmonics you add, overshoot always persists near signal discontinuities. What would it take to eliminate the overshoot completely? Why is it inevitable in Fourier Series?
3. **Perceptual Effect of Harmonics:** As you added harmonics one at a time, how did the perceived tone (timbre) of the sound change? What physical phenomena did your ears detect (e.g., higher pitch, more "buzzing," sharper attack)? Relate this to the changes you saw on the oscilloscope.
4. **Practical Fidelity Trade-Off:** Your target waveform ideally has infinite harmonics, but you only synthesized up to $N = 10$. For a real-world application (e.g., telecommunications, audio synthesis), when would you accept the truncation error shown in your $N = 10$ approximation, and when would you demand more harmonics? Justify with technical reasoning.
5. **Edge Device Feasibility:** Reflect on the memory and CPU constraints of your Arduino (2 KB RAM, 32 KB Flash, 16 MHz clock). How would you estimate the computational cost (time and memory) of synthesizing a signal with $N = 100$ harmonics vs. $N = 10$? What would break first—memory or clock cycles?

Student Notes: Space for discussion answers (or attach separate sheet with name/ID):

8.2 Final Submission Checklist

Before submitting, verify that your submission includes **all of the following**:

- ☐ **Pre-lab work:** Handwritten Fourier Series derivation with name/ID, coefficient tables, and sampling plan.
- ☐ **Hardware photo:** Breadboard setup with low-pass filter, amplifier, button, and oscilloscope probe clearly visible.
- ☐ **Arduino code:** Printed source code showing your Fourier coefficients, harmonic synthesis loop, and button interrupt handler.
- ☐ **Audio/video recording:** 30-second video (saved with name/ID in filename) showing oscilloscope waveform and speaker audio while adding harmonics.
- ☐ **MATLAB Figure 1:** Empirical FFT spectrum of the hardware output with axis labels, legend, and your name/ID in title.
- ☐ **MATLAB Figure 2:** Overlay plot comparing empirical FFT (from hardware) vs. hand-calculated Fourier coefficients (theory).
- ☐ **MATLAB Figure 3:** Gibbs Phenomenon visualization (four subplots with N=1, 3, 5, 10

harmonics), clearly showing ringing and convergence.

- ☐ **Discussion answers:** Written responses to five discussion questions with observations and calculations.
- ☐ **MATLAB code appendix:** Print the key analysis sections (FFT computation, plotting, coefficient overlay).
- ☐ **TA sign-off page:** See Table 5 below, completed with TA initials at each checkpoint.

8.3 TA Checkpoint Sign-Off

Table 5. TA Checkpoint Sign-Off (Complete as Lab Progresses)

CP	Requirement	Target Time	TA Initials & Date
CP-1	Pre-lab derivations and coefficient table verified. Hardware output produces expected DC voltage when tested.	0:00–0:20	_____
CP-2	Arduino successfully synthesizes waveform. Button control works; waveform visibly changes on oscilloscope with each harmonic added. Audio tone evolution heard by TA. 30-second video recorded.	0:20–1:20	_____
CP-3	MATLAB capture successful. FFT spectrum computed and plotted. Overlay of empirical vs. hand-calculated spectrum generated. Gibbs Phenomenon visualization shows four partial sums (N=1, 3, 5, 10).	1:20–2:00	_____

Lab Complete: Once all three checkpoints are signed off and the final submission checklist is complete, your proof-of-concept is accepted. Congratulations—you have proved that a low-cost microcontroller can synthesize arbitrary periodic signals. Your Senior Engineering Lead is impressed.

Appendix A: Fourier Series Examples and Coefficients

A.1 Square Wave (50% Duty Cycle)

For $x(t) = \begin{cases} 1 & 0 < t < T/2 \\ -1 & T/2 < t < T \end{cases}$:

Coefficients:

$$a_0 = 0 \quad \text{(no DC)} \tag{11}$$

$$a_k = 0 \quad \text{(no cosines, by symmetry)} \tag{12}$$

$$b_k = \begin{cases} \frac{4}{k\pi} & k \text{ odd} \\ 0 & k \text{ even} \end{cases} \tag{13}$$

Series:

$$x(t) = \frac{4}{\pi}\sin(\omega_0 t) + \frac{4}{3\pi}\sin(3\omega_0 t) + \frac{4}{5\pi}\sin(5\omega_0 t) + \cdots \tag{14}$$

A.2 Sawtooth Wave

For $x(t) = \frac{2t}{T} - 1$, $0 < t < T$:

Coefficients:

$$a_0 = 0 \tag{15}$$

$$a_k = 0 \quad \text{(odd symmetry)} \tag{16}$$

$$b_k = -\frac{2}{k\pi} \tag{17}$$

Series:

$$x(t) = -\frac{2}{\pi}\sin(\omega_0 t) - \frac{1}{\pi}\sin(2\omega_0 t) - \frac{2}{3\pi}\sin(3\omega_0 t) - \cdots \tag{18}$$

A.3 Triangular Wave

For a symmetric triangle peaking at $t = T/4$:

Coefficients:

$$a_0 = 0 \tag{19}$$

$$a_k = \frac{8}{\pi^2 k^2} \quad k \text{ odd}, \quad a_k = 0 \quad k \text{ even} \tag{20}$$

$$b_k = 0 \quad \text{(even symmetry)} \tag{21}$$

Series:

$$x(t) = \frac{8}{\pi^2}\cos(\omega_0 t) + \frac{8}{9\pi^2}\cos(3\omega_0 t) + \frac{8}{25\pi^2}\cos(5\omega_0 t) + \cdots \tag{22}$$

Appendix B: Arduino PWM and Timer Configuration

B.1 Changing PWM Frequency

The default PWM frequency on D11 is 490 Hz. To increase it for better audio fidelity, reconfigure Timer2:

```
// Set PWM frequency on D11 to ~8 kHz
TCCR2B = TCCR2B & 0b11111000 | 0b00000001;  // prescaler = 1
```

Formula: $f_{PWM} = f_{CPU}/(256 \times \text{prescaler})$. For prescaler = 1: $f_{PWM} = 16$ MHz$/256 = 62.5$ kHz (maximum for 8-bit PWM).

For 8-bit resolution, maximum achievable PWM frequency is 62 kHz. If you need lower (e.g., 8 kHz), use prescaler 8: $f_{PWM} = 16$ MHz$/(256 \times 8) = 7.8$ kHz.

B.2 High-Speed Timer Interrupt

To output samples at a precise sampling rate (e.g., 10 kHz), use Timer1 interrupt:

```
// Initialize Timer1 for sampling rate Fs_Hz
void initSamplingTimer(int Fs_Hz) {
  cli();  // disable interrupts

  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1 = 0;

  // Set compare match register: OCR1A = (16 MHz) / (1 * Fs_Hz) - 1
  OCR1A = (16000000 / Fs_Hz) - 1;

  // Set CTC mode (Clear Timer on Compare Match)
  TCCR1B |= (1 << WGM12);

  // Set prescaler = 1
  TCCR1B |= (1 << CS10);

  // Enable Timer1 compare interrupt
  TIMSK1 |= (1 << OCIE1A);

  sei();  // enable interrupts
}

// Interrupt service routine (called at Fs_Hz rate)
ISR(TIMER1_COMPA_vect) {
  // Output the next sample to PWM
  analogWrite(D11, waveform_samples[sample_index]);

  // Advance to next sample (wrap around at N_SAMPLES)
  sample_index = (sample_index + 1) % N_SAMPLES;
}
```

The skeleton code provides this; you only need to call `initSamplingTimer(FS_HZ)` in setup.

Appendix C: MATLAB Fourier Analysis Templates

C.1 FFT Computation and Positive-Frequency Plotting

```
% Input: samples (1 x N array of ADC values, 0-255 scale)
% Convert to normalized voltage (-1 to +1 range)
samples_normalized = (samples / 127.5) - 1;

% Compute FFT
Y = fft(samples_normalized);
N = length(samples_normalized);

% Compute magnitude spectrum (one-sided)
magnitude = abs(Y(1:N/2)) / (N/2);  % normalize by N/2 for one-sided spectrum
frequency = (0:N/2-1) * Fs / N;      % frequency axis

% Plot
figure;
stem(frequency, magnitude, 'b', 'filled');
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('Magnitude', 'FontSize', 12);
title('Empirical FFT Spectrum', 'FontSize', 12);
grid on;
```

C.2 Overlay: Empirical vs. Hand-Calculated Spectrum

```
% Your hand-calculated Fourier coefficients
a_coeffs = [0, 1.27, 0, 0.42, 0, 0.254, ...];  % cosine
b_coeffs = [0, 0, 0, 0, 0, 0, ...];              % sine

% Magnitude of each harmonic: |c_k| = sqrt(a_k^2 + b_k^2) / 2
magnitude_theory = sqrt(a_coeffs.^2 + b_coeffs.^2) / 2;

% Plot both
figure;
hold on;
stem(frequency, magnitude, 'b', 'filled', 'DisplayName', 'Empirical FFT');
f_theory = (1:length(magnitude_theory)) * F0_HZ;
stem(f_theory, magnitude_theory, 'r*', 'MarkerSize', 12, ...
     'DisplayName', 'Hand-Calculated');
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('Magnitude', 'FontSize', 12);
title('Empirical vs. Theory', 'FontSize', 12);
legend('FontSize', 11);
grid on;
hold off;
```

C.3 Gibbs Phenomenon: Partial-Sum Visualization

```
% Time vector for one period
```

```
T_period = 1 / F0_HZ;
t = linspace(0, T_period, 1000);

% Create four subplots with N = 1, 3, 5, 10 harmonics
figure('Position', [100, 100, 1200, 800]);

for idx = 1:4
    N_harm = [1, 3, 5, 10](idx);

    subplot(2, 2, idx);

    % Synthesize partial sum
    x_partial = a_coeffs(1)/2 * ones(size(t));
    for k = 1:N_harm
        omega_k = 2 * pi * k * F0_HZ;  % angular frequency of k-th harmonic
        x_partial = x_partial ...
            + a_coeffs(k+1) * cos(omega_k * t) ...
            + b_coeffs(k+1) * sin(omega_k * t);
    end

    plot(t * 1000, x_partial, 'b-', 'LineWidth', 1.5);
    xlabel('Time (ms)', 'FontSize', 11);
    ylabel('Amplitude', 'FontSize', 11);
    title(sprintf('N = %d harmonics', N_harm), 'FontSize', 11);
    grid on;
    ylim([-1.5, 1.5]);  % fixed y-range to show overshoot
end

sgtitle('Gibbs Phenomenon: Convergence with Increasing Harmonics', ...
        'FontSize', 12);
```

Appendix D: Troubleshooting and Common Issues

Hardware and Audio Issues

1. **No sound from speaker, or very quiet sound.**
   - Verify the amplifier power supply is on (check voltage at op-amp power pins).
   - Test with a simple DC test: `analogWrite(D11, 128)` should produce a steady voltage and quiet hum from the speaker.
   - Check speaker impedance matching. Some speakers may require a current-limiting resistor (e.g., 8 Ω speaker with 100 Ω series resistor to reduce volume).
   - If oscilloscope shows large signal after filter but speaker is silent, the amplifier may not be powered correctly.
2. **Waveform on oscilloscope looks distorted or clipped.**
   - Check that the PWM duty cycle is within 1–254 (not 0 or 255, which are extremes).
   - Verify the low-pass filter cutoff frequency (1 kHz) is well below your sampling rate. If the cutoff is too low (e.g., 100 Hz), high-frequency harmonics are attenuated.
   - Clipping may indicate your signal amplitudes are too large. Reduce by dividing the waveform samples by 2 in the synthesis code.
3. **Gibbs phenomenon overshoot is very pronounced (¿50% overshoot).**
   - This is normal and expected for square/sawtooth/pulse waves! The Gibbs Phenomenon always overshoots by 9% of the discontinuity magnitude.
   - If overshoot is much larger, your signal may be clipping or your coefficients may be wrong.
4. **Tone does not change when you add harmonics.**
   - Check that your Fourier coefficients for higher harmonics ($k > 1$) are non-zero. If they are all zero, only the fundamental is being synthesized.
   - Verify the button interrupt is firing: add a Serial print statement in the `buttonISR()` to confirm `num_harmonics` is incrementing.
   - Ensure `regenerateWaveform()` is being called and the PWM output is being updated.

Arduino Code Issues

1. **Arduino does not respond to button press.**
   - Verify the button is connected to D2 (INT0 interrupt pin) and the other side to GND.
   - Use a multimeter to confirm the button makes a connection when pressed.
   - Add a 10 kΩ pull-up resistor to D2 (internal pull-up via `pinMode(D2, INPUT_PULLUP)`).
   - Check the interrupt handler is properly registered: `attachInterrupt(digitalPinToInterrupt(2), buttonISR, FALLING);`.
2. **Synthesis loop takes too long; audio glitches or stutters.**

- Avoid floating-point math inside the timer interrupt if possible. Use fixed-point or pre-computed lookup tables.
- Reduce the number of harmonics you're computing in real-time. Pre-compute the full waveform in setup(), then just update the samples array.
- If `regenerateWaveform()` is being called in the button interrupt, keep it simple to avoid blocking the timer.

3. **Compile error: "multiple definitions of ISR".**
   - You've accidentally defined the same interrupt handler twice. Check your code for duplicate `ISR(TIMER1_COMPA_vect)` or `ISR(INT0_vect)`.
   - If using the skeleton, ensure you're not adding your own interrupt handlers where the skeleton already defines them.

MATLAB Analysis Issues

1. **FFT plot shows very small magnitudes (e.g., all peaks < 0.01).**
   - Check your normalization. The formula for one-sided spectrum is: magnitude $= |Y(k)|/(N/2)$ for $k = 1$ to $N/2 - 1$.
   - Verify your input samples are in the correct range (e.g., 0–255 for 8-bit ADC). If they're already normalized to ± 1, use a different normalization.
   - Ensure you're using the correct sampling rate: $f = (\text{sample index}) \times f_s/N$.
2. **Empirical FFT doesn't match hand-calculated coefficients.**
   - Double-check your hand calculations. Symmetry errors (e.g., missing negative coefficients for exponential form) are common.
   - Check your Arduino code: are the coefficients correctly entered? Print them to serial for verification.
   - Noise and ADC quantization will cause small deviations. These are acceptable and expected.
3. **Gibbs Phenomenon plot looks strange (no overshoot visible).**
   - Ensure your target waveform has a discontinuity (e.g., square wave, sawtooth). Smooth waveforms don't exhibit Gibbs ringing.
   - Check that your Fourier coefficients are correct. If they're all near-zero, the reconstructed signal will be nearly flat.
   - Verify the y-axis limits are wide enough to show overshoot: use `ylim([-1.5, 1.5])` for a signal ideally in $[-1, 1]$.

When to Ask for TA Help

- If the low-pass filter output is still showing the 490 Hz PWM ripple (suggests filter cutoff is too high or components are wrong).
- If the button interrupt is not responding after 10 minutes of debugging.

- If your FFT output shows nonsensical values (e.g., massive peaks at very low frequencies).
- If you suspect your hand-calculated Fourier coefficients are wrong (compare with reference table in Appendix A).

References and Inspiration

This laboratory demonstrates that any periodic sound can be built from simple sine waves—a central principle of audio engineering and music synthesis across decades of innovation.

University Course Inspirations

Georgia Tech ECE 2026: Real-Time Audio Synthesis**

Lab 07 is directly inspired by Georgia Tech's flagship signals and systems course, which emphasizes that Fourier Series is not an abstract mathematical artifact but the foundation of all music synthesis, audio effects, and sound design. Georgia Tech's approach of implementing harmonic synthesis on embedded hardware and hearing the results immediately builds intuition for the abstract mathematics.

MIT 6.003: Fourier Series and Frequency Domain Analysis

MIT's rigorous treatment of Fourier Series—both trigonometric and exponential forms—and the connection to frequency-domain representation of signals provides the theoretical foundation. MIT's emphasis on deriving Fourier coefficients by hand informs this lab's pre-lab calculation requirements.

Georgia Tech's Audio DSP Labs

The use of microcontroller-based sound synthesis and real-time harmonic manipulation reflects Georgia Tech's commitment to making the abstract concept of harmonic decomposition tangible through audio that students can hear directly.

Rice University ELEC 241: Signal Generation and Real-Time Processing

Rice's approach to implementing signal generation on embedded systems and the integration of hardware synthesis with Simulink modeling informs the Model-Based Design philosophy of this lab.

Duke University ECE 280: Lab Modernization

This lab is part of Duke's curriculum redesign emphasizing that Fourier Series is the bridge between time-domain waveforms (what you see on an oscilloscope) and frequency-domain understanding (what you hear from a speaker).

References

- A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd ed. Pearson, 1997. Chapter 3: Fourier Series representation of periodic signals.
- A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010. Chapter on Fourier analysis.
- J. H. McClellan, R. W. Schafer, and M. A. Yoder, *DSP First: A Multimedia Approach*, 2nd ed. Pearson, 2015. Chapters on Fourier Series and synthesis.
- K. C. Pohlmann, *Principles of Digital Audio*, 6th ed. McGraw-Hill, 2011. Chapter on harmonic analysis and synthesis in audio.